

MACHINE LEARNING CAPSTONE PROJECT

Predicting Flight Arrival Delay from Post Departure Information

Khalid Al Dosari • Abdulaziz Alshareef • Anas Alzahrani • Feras Madkhali

Tuwaiq Bootcamp

1 Problem and Dataset

Objective. Predict how late a US domestic flight will arrive, at the moment it pushes back from the gate. Two targets are modelled on the same rows, the same features and the same split: **ArrDel15** (classification, does the flight land 15 minutes late or more) and **ArrDelay** (regression, arrival delay in minutes, which may be negative when a flight arrives early).

Real world motivation. Once an aircraft has left the gate, airlines, airports and connecting passengers need an arrival estimate sharper than the printed schedule. Gate assignment, crew rotation and passenger connection decisions all depend on it.

Dataset. Kaggle, *Flight Delay Dataset 2018–2022*, file **Combined_Flights_2018.csv**, originally published by the US Department of Transportation, Bureau of Transportation Statistics (On Time Performance database).

https://www.kaggle.com/datasets/robikscube/flight-delay-dataset-20182022?select=Combined_Flights_2018.csv

One row is one scheduled domestic flight. The raw file contains **5,689,512 rows** and **61 columns**, mixing numerical measurements, categorical identifiers and schedule fields.

Why machine learning is appropriate. Arrival delay depends on how late the aircraft already is, ground congestion, route, carrier and time of day, all interacting. No closed form rule captures this, but a model can learn the pattern from millions of historical flights.

2 Data Quality and Cleaning

Seven checks were run on the raw data. The decision taken mattered more than the technique used, so each is justified rather than applied mechanically.

Check	What was found	Decision and reason
Missing values	1.8% of arrival fields, 1.5% of departure fields	Drop cancelled (1.55%) and diverted (0.25%) flights. A flight that never arrived has no arrival delay to predict, so imputing would invent data.
Duplicate rows	None found	Nothing to remove.
Incorrect types	<code>CRSDepTime</code> is an integer but encodes a clock time (1259 to 1300 is one minute, not 41)	Convert to minutes since midnight, then to cyclic sine and cosine so 23:59 and 00:01 sit close together.
Invalid values	67 rows with negative scheduled or actual duration	Drop. Physically impossible and unrecoverable; 67 of 5.69M is negligible.
Inconsistent categories	<code>Airline</code> , <code>Origin</code> , <code>Dest</code> checked for case and spelling variants, none collide	No cleaning required.
Outliers	IQR flags 9.4% of <code>ArrDelay</code> and 13.1% of <code>DepDelay</code> ; skewness 8.5 and 9.6	Keep all. The long right tail is the phenomenon the project exists to predict. Removing it would bias the model toward underestimating real delays.
Class imbalance	80.5% on time against 19.5% delayed	Report ROC-AUC, Precision, Recall and F1 rather than Accuracy alone; use <code>class_weight="balanced"</code> where supported.

After cleaning, **5,578,560 rows** remain.

3 Exploratory Analysis: Five Findings

1. **Departure delay dominates.** `DepDelay` correlates with `ArrDelay` at **0.96**. Once a flight leaves late it lands late almost as a rule. This single relationship sets both the ceiling and the floor for the entire project.
2. **Delay compounds through the day.** Mean arrival delay rises from about -3 minutes for 5am departures to $+12$ minutes around 6pm, as aircraft rotations fall progressively behind schedule. Scheduled hour is therefore a cheap and strong feature.
3. **Delay is seasonal.** Summer (June to August, 9.7 to 11.5 minutes mean) is roughly three times worse than autumn (September to October, about 3 minutes), consistent with thunderstorm season. Calendar month carries real signal.
4. **Carriers differ substantially at the same scale.** Among the ten largest carriers, JetBlue is delayed 27.3% of the time against Delta at 12.5%. Operating practice and hub congestion matter, justifying `Airline` as a feature despite being categorical.
5. **Ground congestion adds independent signal.** `TaxiOut` correlates with `ArrDelay` at 0.22, moderate but real, and is not visible in `DepDelay` alone. Conversely `Distance` and `CRSElapsedTime` correlate at 0.98 with each other but only -0.02 with the target, so trip length alone says little about lateness and only one of the two is retained.

4 Feature Preparation and the Leakage Boundary

The single most consequential decision in this project is *when* the prediction is made. We fix that moment at departure, which determines exactly which columns are legal.

Legal at departure (used)	Banned, known only after landing
DepDelay, TaxiOut, CRSElapsedTime, dep_sin, dep_cos, Month, DayofMonth, DayOfWeek, is_weekend, Airline, Origin, Dest	ArrTime, WheelsOn, TaxiIn, ActualElapsedTime, AirTime, ArrDelayMinutes, ArrivalDelayGroups, DivAirportLandings

Enforcement. The feature list is asserted against the banned list in code before any model is fitted, and the train/validation/test split happens before any target dependent statistic is computed. No test data is used to fit scalars, encoders or models.

Transformations and their justification.

- Identifier and near duplicate columns removed: they carry no independent signal or duplicate CRSElapsedTime.
- Cyclic encoding of scheduled departure time: required so that midnight wraps correctly. Necessary for distance and margin based models (KNN, SVM) and linear models; harmless for trees.
- One hot encoding of Airline, Origin, Dest with a minimum frequency threshold, so rare airports do not each receive a column.
- Standardisation of numeric columns for scale sensitive models (Logistic and Linear Regression, KNN, SVM). Tree based models receive raw values, since they split on thresholds and are scale invariant.

Sampling and splitting. SVM and SVR scale roughly quadratically to cubically with row count and become impractical beyond a few tens of thousands of rows. Rather than evaluate different models on different samples, one stratified sample of **30,000 rows** is shared by every model so comparisons remain fair. It is split **60/20/20** into train (18,000), validation (6,000) and test (6,000), stratified on ArrDel15 at every step so the 19.5% delay rate is preserved throughout. The test set is evaluated once, for the final reported numbers only.

5 Models and Results

Four algorithms were trained per problem, chosen to span different learning styles: linear (baseline), instance based, a boosted ensemble and a margin based model.

Classification (target ArrDel15)

Model	Train AUC	Val AUC	Test AUC	Test F1	Gap
Logistic Regression (baseline)	0.968	0.957	0.9649	0.821	0.011
K-Nearest Neighbors	0.941	0.898	0.8969	0.559	0.043
Gradient Boosting (XGBoost)	0.993	0.957	0.9631	0.835	0.036
Support Vector Machine (RBF)	0.985	0.956	0.9639	0.831	0.030

Regression (target ArrDelay, minutes)

Model	Train MAE	Val MAE	Test MAE	Test RMSE	Test R^2
Linear Regression (baseline)	7.17	7.35	7.29	10.35	0.958
KNN Regressor	11.83	12.27	12.54	18.36	0.868
Gradient Boosting Regressor	6.60	8.40	8.29	19.18	0.856
Support Vector Regression	8.14	9.20	9.39	31.31	0.615

Why ROC-AUC and not Accuracy. With a 80/20 split, a model predicting “on time” for every flight scores about 80% Accuracy while catching zero delayed flights. ROC-AUC is threshold independent and undistorted by the imbalance, so it is the primary classification metric. RMSE is reported alongside MAE because it penalises large errors more heavily, and the largest errors are the operationally important ones.

6 Overfitting and Hyperparameter Behaviour

Using the rule that a train minus validation gap below 0.02 generalises well and above 0.05 is substantial, no model is severely overfit at its chosen settings. Logistic Regression is the tightest (0.011); XGBoost is the loosest of the non-KNN models (0.036, train AUC 0.993 against validation 0.957), indicating it has begun to fit training noise. KNN shows the weakest performance overall, hurt by the high dimensional one hot space where distance loses meaning.

Three hyperparameter sweeps were run, and all three produced the same shape: validation performance rises to a peak and then declines while training performance keeps climbing.

Model	Parameter	Behaviour observed	Chosen
KNN	<code>n_neighbors</code>	$k=1$ memorises: train 1.000, val 0.707, gap 0.293. Gap falls to 0.016 by $k=100$.	$k = 50$
XGBoost	<code>learning_rate</code>	Validation peaks at 0.03 (0.961); gap grows 0.012 $\rightarrow$ 0.052 as rate rises to 0.40.	0.03
SVM	<code>C</code>	Validation peaks at $C=1$ (0.956); gap reaches 0.060 by $C=100$.	$C = 1$

The lesson is consistent: model complexity must be tuned against a validation set, never selected by what looks strongest on training data.

7 Model Selection

Selected classifier: Logistic Regression. It attains the best test ROC-AUC (0.9649, marginally ahead of XGBoost at 0.9631), has the smallest train-validation gap of all four models, trains in a fraction of a second, and produces directly interpretable coefficients, which matters for an operational tool where staff may need to know why a flight was flagged. XGBoost achieves a slightly higher F1 (0.835 against 0.821), but that gain does not justify the added complexity and the larger overfitting gap.

Selected regressor: Linear Regression. It records the best test R^2 and MAE of all four models, beating every more complex alternative outright. This is coherent with the problem structure: `ArrDelay` is close to a linear function of `DepDelay` plus a smaller adjustment, so a linear model already captures most of the signal, and additional flexibility costs generalisation and speed without buying accuracy.

Selection considered performance, generalisation, interpretability and computational cost together, not score alone. Notably, neither advanced model clearly beat its baseline: for regression the baseline is the outright winner.

8 Error Analysis: Three Findings

Confusion matrix for the selected classifier on the 6,000 row test set: TN 4,502, FP 329, FN 125, TP 1,044.

- False negatives are flights that departed on time.** The 125 missed delays have a mean `DepDelay` of just 0.15 minutes, effectively identical to correctly predicted on time flights, against 71.6 minutes for true positives. These flights lost their time *in the air*, after the moment the model makes its prediction. No feature available at departure could have caught them. This is a hard limit of the framing, not a fixable modelling weakness.
- False positives are genuinely borderline.** The 329 false alarms have a mean `DepDelay` of 12.3 minutes and `TaxiOut` of 22.6, both clearly above the correct negative averages (-3.2 and 15.4). These are flights that looked risky at departure and then recovered time en route, an uncertain case rather than a model error.
- Regression residuals are mildly but systematically biased.** For on time flights the mean residual is -2.6 minutes (slight over prediction), while for *every* bucket of actually delayed flights it is consistently positive at $+4$ to $+6$ minutes. The model under predicts how severe a delay becomes once one is underway. The largest individual errors follow the same in air disruption pattern as the classification false negatives.

Which error matters more depends on use. False negatives (a passenger told the flight is fine when it is not) are more damaging operationally than false positives, which argues for keeping the balanced class weighting that favours Recall on the delayed class.

9 Unsupervised Learning

Clustering. K-Means and DBSCAN were applied to six flight shape features (CRSElapsedTime, Distance, dep_sin, dep_cos, DepDelay, TaxiOut), standardised because both methods rely on distance. **The target was never used.**

K-Means used $k=4$: silhouette technically peaks at $k=2$ (0.323) but produces one dominant split, while $k=4$ (0.286) yields interpretable, differently sized segments.

- **Cluster 0** (4,599): long haul, 277 min and 1,935 mi, on time on average, 18% delayed.
- **Cluster 1** (10,222): short flights departing earlier in the day, 24% delayed, the worst of the three normal clusters.
- **Cluster 2** (14,567): short and medium flights departing later, the calmest group at 14% delayed.
- **Cluster 3** (612): mean ArrDelay **245.8 minutes**, 100% delayed. K-Means isolated a severe delay segment entirely on its own, from schedule and ground features alone, without ever seeing the target.

DBSCAN used `min_samples = 12` (twice the six clustering features, a standard rule of thumb) and `eps = 0.90`, chosen from the knee of the k-distance plot (distance to each point's 12th nearest neighbour, sorted). At these settings DBSCAN does not partition the data into several groups at all: it finds one dense core cluster (29,299 flights, 97.7%, mean ArrDelay 1.4 minutes, 18% delayed) and marks **701 flights (2.3%)** as noise, points too sparse to belong to any dense neighbourhood. That noise set has mean ArrDelay 185.4 minutes, mean TaxiOut 39.1 minutes against 16.6 in the core, and a 92% delay rate, essentially the same severely delayed segment K-Means isolated, reached without ever specifying how many clusters to look for. Agreement is partial rather than exact: 52.6% of K-Means' 612 flight cluster is labelled noise by DBSCAN, and 45.9% of DBSCAN's noise falls inside that K-Means cluster. The two algorithms disagree at the margin because they define "unusual" differently, K-Means by distance to a centroid, DBSCAN by local density.

PCA. One hot encoding produces **220 features**. Retaining 90% of the variance requires **44 principal components**, a five fold reduction. Logistic Regression trained on the reduced representation scores test ROC-AUC 0.9640 against 0.9649 on the full feature set, a difference of 0.0009. PCA is therefore useful here as *compression* rather than as an accuracy improvement: valuable for a slower model such as KNN or SVM, or a storage constrained deployment, but unnecessary for performance, since Logistic Regression already handles 220 features without overfitting. Projected onto the first two components the classes overlap heavily, which is expected, as those two components capture only a modest share of total variance.

10 Conclusions and Recommendations

Most important finding. The 0.96 correlation between departure and arrival delay both enables this project and bounds it.

Highest impact decision. Fixing the leakage boundary at "moment of departure" rather than "moment of landing". That choice, made in Part 1 before any modelling, influenced the final numbers more than any algorithm or hyperparameter selected afterwards. A strictly pre departure model on the same data would top out far lower.

Limitations.

- The model is only usable *after* a flight has departed. It cannot address the harder and more valuable pre departure problem.
- It cannot observe weather or air traffic control conditions en route, which is precisely where its residual errors originate.
- Results are based on a 30,000 row sample of a single year (2018), constrained by SVM scalability.

With more time or data. Add live weather along the route to attack the in air blind spot; build a strictly pre departure model side by side so the value of waiting until departure becomes directly measurable rather than argued; and refit the selected linear models on the full 5.58M rows, which is computationally trivial once SVM is dropped.

Recommendation for real world use. Yes, for post departure updates: passenger notification, gate assignment and connection planning, given strong and well generalising test performance. No, for any decision that must be made before departure, which is a different and substantially harder problem this project deliberately did not attempt.

11 Reflection

The most challenging part was deciding precisely where the leakage boundary sits. It is easy to state that no post landing column may be used, but `TaxiOut` and `WheelsOff` sit exactly on the line and required an explicit, defensible decision about when “now” is.

The most surprising result was that the baselines won. Linear Regression beat every more complex regressor outright, and Logistic Regression matched or beat every more complex classifier on ROC-AUC while generalising better than all of them. Additional complexity did not translate into a better model.

On model selection, the best model is not the most sophisticated one; it is the one that generalises reliably and fits the deployment constraints of speed and interpretability. On overfitting, every sweep in Part 11 showed the same rise then fall in validation performance, which reframed overfitting from a binary failure state into a matter of degree that must be checked at every complexity setting.

If repeated, we would build the pre departure model in parallel from the start, so the two framings could be compared directly rather than one being argued relative to the other.